BioBatch Sentinel

Design document · RL environment for synthetic bio-monitoring code tasks

1. Purpose and goals

BioBatch Sentinel is a tool-using coding environment where an agent inspects a small Python project, edits code, runs checks, and explicitly finishes the task. Five deterministic synthetic tasks inspired by sensor/process monitoring test instruction following, code modification, validation, and termination. These are software exercises—not operational guidance for a real bioprocess.

2. Task suite

Task	Level	Expected behavior
sensor_calibration	Easy	Subtract offset before multiplying by scale.
ph_alarm	Easy	Apply strict out-of-range boundary checks.
do_trend	Medium	Average valid dissolved-oxygen values, ignore None, return 0.0 if none are valid.
contamination_signal	Medium	Use an inclusive threshold for the signal.
recovery_action	Medium	Priority: contamination, low DO, pH alarm, then monitoring.

3. Runtime architecture

Docker Compose separates the harness, trusted sandbox control service, and executor worker. The harness coordinates episodes, model calls, tools, verification, rewards, and logs. The control service handles reset, workspace access, and verification. The executor runs submitted Python snippets in a separate container configured with network_mode: none, exchanging requests through a dedicated shared queue volume. This is defense-in-depth, not a claim of perfect isolation; workspace permissions and container configuration remain security-sensitive.

4. Episode lifecycle

A seeded reset initializes task state and workspace. The model receives instructions and tool schemas, then submits structured actions such as file reads/writes, code execution, or finish. State persists across steps. On finish or a cap, the programmatic verifier evaluates tests and regressions, computes reward, and records termination reason.

BioBatch Sentinel · Design

5. Verification and baseline gate

Each task has a reference solution (oracle) and a no-op baseline. The gate requires oracle ≥ 0.95 and no-op ≤ 0.05. The recorded gate passed: all five oracle scores were 1.00 and all no-op scores were 0.00. The recorded full model run solved all five tasks with mean reward 1.00; each succeeded in five steps. A separate mock/idle run finished without edits and received reward 0.00, supplying a genuine failure trajectory.

6. Logs and evidence

JSONL logs record task/seed, model input and raw output, parsed actions, tool results, errors, state hash, reward, tokens, and latency. Episode-end records include steps, termination, reward, and elapsed time. Evidence assembled for submission includes baseline-results/gate.json, five task logs, and success/failure sample trajectories. Review logs for accidental secrets before sharing.

7. Reproducibility and limitations

Intended launch command: `docker compose up --build --abort-on-container-exit --exit-code-from harness`. Use a placeholder-only .env.example and never include the real .env/API key in a submission. Repeat the baseline and full-task runs after final changes. The tasks are small and synthetic; they do not establish suitability for real laboratory or industrial control. The executor network check showed that it could not resolve the sandbox hostname, but this is not a formal security proof.